

Phần 4: Spring JPA Integration

4.1 Spring Boot Auto-Configuration

Spring Boot tự động cấu hình JPA dựa trên các dependency và properties trong `application.properties` hoặc `application.yml`.

4.1.1 Dependencies cần thiết

Để sử dụng Spring Data JPA trong dự án Spring Boot, bạn cần thêm các dependency sau vào file `pom.xml` (nếu dùng Maven) hoặc `build.gradle` (nếu dùng Gradle):

XML

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId> <!-- Dependency
chính để sử dụng Spring Data JPA, bao gồm cả Hibernate và JPA API. -->
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope> <!-- Database trong bộ nhớ, thường dùng cho
phát triển và test. -->
  </dependency>
  <!-- Hoặc database khác như MySQL, PostgreSQL -->
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope> <!-- Driver kết nối đến MySQL. Bạn sẽ cần thay
thế bằng driver phù hợp với database bạn sử dụng. -->
  </dependency>
</dependencies>
```

Giải thích:

- `spring-boot-starter-data-jpa`: Đây là "starter" chính cho Spring Data JPA. Nó tự động cấu hình Hibernate (triển khai JPA mặc định của Spring Boot) và các thành phần cần thiết khác.

- `h2` : Một database trong bộ nhớ, rất tiện lợi cho việc phát triển và chạy thử nghiệm mà không cần cài đặt database riêng.
- `mysql-connector-java` : Driver JDBC để kết nối ứng dụng Java với cơ sở dữ liệu MySQL. Bạn sẽ cần thay thế hoặc bổ sung các driver tương ứng nếu sử dụng các hệ quản trị cơ sở dữ liệu khác như PostgreSQL (`postgresql`), Oracle (`ojdbc8`), v.v.

4.1.2 Cấu hình Database trong `application.properties`

File `application.properties` là nơi bạn cấu hình các thuộc tính cho ứng dụng Spring Boot, bao gồm cả thông tin kết nối cơ sở dữ liệu và các thuộc tính JPA/Hibernate.

Plain Text

```
# H2 Database (for development) - Cấu hình cho cơ sở dữ liệu H2 trong bộ nhớ,
thường dùng cho môi trường phát triển và kiểm thử.
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE # URL kết nối đến H2. DB_CLOSE_DELAY=-1 giữ cho DB không đóng khi kết
nối cuối cùng bị ngắt.
spring.datasource.driverClassName=org.h2.Driver # Tên driver JDBC cho H2.
spring.datasource.username=sa # Tên người dùng cho DB.
spring.datasource.password=password # Mật khẩu cho DB.
spring.h2.console.enabled=true # Bật H2 Console để xem và quản lý DB qua
trình duyệt.

# MySQL Database - Cấu hình cho cơ sở dữ liệu MySQL.
spring.datasource.url=jdbc:mysql://localhost:3306/mydb?
useSSL=false&serverTimezone=UTC # URL kết nối đến MySQL. useSSL=false tắt
SSL, serverTimezone=UTC thiết lập múi giờ.
spring.datasource.username=root # Tên người dùng cho DB MySQL.
spring.datasource.password=password # Mật khẩu cho DB MySQL.
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver # Tên driver
JDBC cho MySQL.

# JPA/Hibernate Properties - Các thuộc tính cấu hình cho JPA và Hibernate.
spring.jpa.hibernate.ddl-auto=update # ddl-auto: update sẽ cập nhật schema
của DB dựa trên các Entity. Các giá trị khác: create, create-drop, validate,
none.
spring.jpa.show-sql=true # Hiển thị các câu lệnh SQL mà Hibernate tạo ra trên
console.
spring.jpa.properties.hibernate.format_sql=true # Định dạng các câu lệnh SQL
hiển thị cho dễ đọc.
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect #
Chỉ định dialect của Hibernate cho MySQL 8, giúp Hibernate tạo ra SQL tối ưu
cho DB này.
```

```
# Connection Pool (HikariCP) - Cấu hình cho HikariCP, một connection pool mặc định của Spring Boot, giúp quản lý kết nối DB hiệu quả.  
spring.datasource.hikari.maximum-pool-size=20 # Số lượng kết nối tối đa trong pool.  
spring.datasource.hikari.minimum-idle=5 # Số lượng kết nối tối thiểu luôn được giữ trong pool.
```

Giải thích:

- **spring.datasource.*** : Các thuộc tính này dùng để cấu hình thông tin kết nối đến cơ sở dữ liệu (URL, username, password, driver). Spring Boot sẽ tự động tạo một **DataSource** bean dựa trên cấu hình này.
- **spring.jpa.hibernate.ddl-auto** : Thuộc tính quan trọng để kiểm soát cách Hibernate tương tác với schema cơ sở dữ liệu. Các giá trị phổ biến:
 - **none** : Không làm gì với schema.
 - **update** : Cập nhật schema hiện có để phù hợp với các Entity. Đây là lựa chọn phổ biến trong phát triển.
 - **create** : Tạo mới schema mỗi khi ứng dụng khởi động (xóa dữ liệu cũ).
 - **create-drop** : Tạo mới schema khi khởi động và xóa khi ứng dụng dừng.
 - **validate** : Kiểm tra xem schema hiện có có khớp với các Entity không, nếu không thì báo lỗi.
- **spring.jpa.show-sql** và **spring.jpa.properties.hibernate.format_sql** : Hữu ích cho việc debug, giúp bạn thấy được các câu lệnh SQL mà JPA/Hibernate thực thi.
- **spring.jpa.properties.hibernate.dialect** : Cần thiết để Hibernate biết cách tạo ra các câu lệnh SQL tối ưu và phù hợp với từng loại cơ sở dữ liệu cụ thể.
- **spring.datasource.hikari.*** : Cấu hình cho HikariCP, một thư viện quản lý kết nối cơ sở dữ liệu hiệu quả, giúp cải thiện hiệu suất ứng dụng.

4.1.3 Cấu hình bằng application.yml

`application.yml` là một định dạng cấu hình thay thế cho `application.properties`, sử dụng cú pháp YAML (YAML Ain't Markup Language) dễ đọc hơn, đặc biệt với các cấu hình phân cấp.

YAML

```
spring:
  datasource: # Cấu hình nguồn dữ liệu
    url: jdbc:mysql://localhost:3306/mydb # URL kết nối đến cơ sở dữ liệu
  MySQL
    username: root # Tên người dùng cơ sở dữ liệu
    password: password # Mật khẩu cơ sở dữ liệu
    driver-class-name: com.mysql.cj.jdbc.Driver # Tên lớp driver JDBC
    hikari: # Cấu hình HikariCP connection pool
      maximum-pool-size: 20 # Số lượng kết nối tối đa
      minimum-idle: 5 # Số lượng kết nối tối thiểu luôn được giữ

  jpa: # Cấu hình JPA/Hibernate
    hibernate:
      ddl-auto: update # Chiến lược tạo/cập nhật schema tự động
      show-sql: true # Hiển thị SQL trên console
      properties:
        hibernate:
          format_sql: true # Định dạng SQL cho dễ đọc
          dialect: org.hibernate.dialect.MySQL8Dialect # Dialect Hibernate cho
MySQL 8
```

Giải thích: Cấu hình trong `application.yml` tương đương với `application.properties` nhưng sử dụng cú pháp YAML. Cấu trúc phân cấp của YAML giúp tổ chức các thuộc tính một cách rõ ràng và dễ đọc hơn, đặc biệt khi có nhiều thuộc tính liên quan đến nhau.

4.2 @Repository Annotation

`@Repository` là một stereotype annotation đánh dấu class là Data Access Object (DAO). Spring sẽ tự động phát hiện và đăng ký các class này như Spring beans, đồng thời cung cấp tính năng dịch ngoại lệ (exception translation) từ các ngoại lệ cụ thể của JPA (như `PersistenceException`) sang các ngoại lệ trừu tượng của Spring (`DataAccessException`).

Java

```
@Repository // Đánh dấu lớp này là một Spring Repository, cho phép Spring
quản lý nó như một bean và áp dụng tính năng dịch ngoại lệ.
public class UserRepositoryImpl {
```

```

    @PersistenceContext // Inject EntityManager vào lớp này. EntityManager là
    giao diện chính để tương tác với Persistence Context trong JPA.
    private EntityManager entityManager;

    public User findByEmail(String email) {
        // Tạo một truy vấn JPQL để tìm User theo email.
        TypedQuery<User> query = entityManager.createQuery(
            "SELECT u FROM User u WHERE u.email = :email", User.class);
        query.setParameter("email", email); // Đặt giá trị cho tham số email.
        return query.getSingleResult(); // Trả về một kết quả duy nhất. Nếu
        không tìm thấy hoặc tìm thấy nhiều hơn một, sẽ ném ngoại lệ.
    }

    public List<User> findActiveUsers() {
        // Tạo một truy vấn JPQL để tìm tất cả User có status là ACTIVE.
        return entityManager.createQuery(
            "SELECT u FROM User u WHERE u.status = 'ACTIVE'", User.class)
            .getResultList(); // Trả về danh sách các User thỏa mãn điều
        kiện.
    }
}

```

Giải thích:

- `@Repository` không chỉ giúp Spring nhận diện các lớp DAO mà còn kích hoạt tính năng dịch ngoại lệ của Spring. Điều này có nghĩa là các ngoại lệ cấp thấp từ JPA (ví dụ: `javax.persistence.NoResultException`) sẽ được chuyển đổi thành các ngoại lệ thân thiện hơn của Spring (ví dụ: `org.springframework.dao.EmptyResultDataAccessException`), giúp code của bạn độc lập hơn với công nghệ persistence cụ thể.
- `@PersistenceContext` được sử dụng để inject một instance của `EntityManager`. Spring sẽ quản lý `EntityManager` này, đảm bảo nó được tạo và đóng đúng cách, và an toàn cho môi trường đa luồng (thread-safe).

4.3 @PersistenceContext

`@PersistenceContext` là một annotation của JPA được sử dụng để inject một instance của `EntityManager` vào một Spring bean. Spring sẽ quản lý lifecycle của `EntityManager` và đảm bảo nó là thread-safe trong môi trường đa luồng.

```

@Repository
public class CustomUserRepository {

    @PersistenceContext // Inject EntityManager mặc định (từ Persistence Unit
    mặc định).
    private EntityManager entityManager;

    @PersistenceContext(unitName = "secondaryPU") // Inject EntityManager từ
    một Persistence Unit cụ thể (khi có nhiều Persistence Unit).
    private EntityManager secondaryEntityManager;

    public void saveUser(User user) {
        entityManager.persist(user); // Sử dụng EntityManager để lưu đối
        tượng User.
    }

    public User findById(Long id) {
        return entityManager.find(User.class, id); // Sử dụng EntityManager
        để tìm đối tượng User theo ID.
    }
}

```

Giải thích:

- `EntityManager` là giao diện chính trong JPA để tương tác với Persistence Context, nơi các Entity được quản lý. Nó cung cấp các phương thức để thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên các Entity.
- Khi bạn sử dụng `@PersistenceContext`, Spring sẽ tự động cung cấp một `EntityManager` instance đã được cấu hình sẵn. Trong môi trường ứng dụng web, `EntityManager` thường được liên kết với vòng đời của một transaction hoặc một request, đảm bảo rằng các thao tác cơ sở dữ liệu được thực hiện trong một ngữ cảnh nhất quán.
- `unitName`: Nếu bạn có nhiều Persistence Unit (ví dụ: kết nối đến nhiều cơ sở dữ liệu khác nhau), bạn có thể chỉ định `unitName` để chọn `EntityManager` từ Persistence Unit cụ thể.

4.4 Repository Interfaces

Spring Data JPA cung cấp các interface repository cơ bản:

4.4.1 Repository<T, ID>

Interface marker cơ bản nhất trong Spring Data. Nó không cung cấp bất kỳ phương thức nào, mà chỉ đóng vai trò là một interface đánh dấu để Spring Data có thể phát hiện các interface repository của bạn.

Java

```
public interface UserRepository extends Repository<User, Long> {  
    // Chỉ định nghĩa các method cần thiết. Spring Data sẽ tự động tạo triển  
    khai cho các method này dựa trên tên của chúng.  
    User findByEmail(String email); // Tìm User theo email.  
    List<User> findByStatus(String status); // Tìm danh sách User theo trạng  
    thái.  
}
```

Giải thích:

- `Repository<T, ID>` là interface gốc cho tất cả các Spring Data repository. `T` là kiểu của Entity mà repository này quản lý, và `ID` là kiểu của khóa chính của Entity đó.
- Mặc dù nó không có phương thức nào, nhưng việc kế thừa từ `Repository` cho phép Spring Data nhận diện interface của bạn là một repository và tạo ra một triển khai (implementation) tại thời điểm chạy (runtime).
- Các phương thức như `findByEmail` và `findByStatus` được gọi là "Query Methods" hoặc "Derived Query Methods". Spring Data JPA có khả năng tự động tạo các truy vấn SQL dựa trên tên của phương thức. Đây là một trong những tính năng mạnh mẽ nhất của Spring Data JPA, giúp giảm đáng kể lượng code boilerplate.

4.4.2 CrudRepository<T, ID>

Kế thừa từ `Repository`, `CrudRepository` cung cấp một tập hợp các phương thức CRUD (Create, Read, Update, Delete) cơ bản cho Entity mà nó quản lý. Đây là một trong những interface được sử dụng phổ biến nhất.

Java

```
public interface UserRepository extends CrudRepository<User, Long> {  
    // Kế thừa các method sau từ CrudRepository:  
    // save(S entity): Lưu một Entity mới hoặc cập nhật một Entity đã tồn  
    tại.  
    // saveAll(Iterable<S> entities): Lưu hoặc cập nhật nhiều Entity cùng
```

```

lúc.
// findById(ID id): Tìm một Entity theo ID. Trả về Optional<T>.
// existsById(ID id): Kiểm tra xem một Entity có tồn tại với ID đã cho
hay không.
// findAll(): Trả về tất cả các Entity.
// findAllById(Iterable<ID> ids): Trả về tất cả các Entity có ID nằm
trong danh sách đã cho.
// count(): Đếm tổng số Entity.
// deleteById(ID id): Xóa một Entity theo ID.
// delete(T entity): Xóa một Entity.
// deleteAll(): Xóa tất cả các Entity.

// Có thể thêm custom methods của riêng bạn
List<User> findByStatus(String status); // Ví dụ về một Query Method tùy
chỉnh.
}

```

Giải thích:

- `CrudRepository` giúp bạn tránh phải viết các phương thức CRUD lặp đi lặp lại cho mỗi repository. Nó cung cấp các thao tác cơ bản nhất để tương tác với cơ sở dữ liệu.
- Các phương thức như `save`, `findById`, `delete` đã được định nghĩa sẵn, bạn chỉ cần kế thừa interface này và Spring Data JPA sẽ tự động cung cấp triển khai cho chúng.
- Bạn vẫn có thể định nghĩa thêm các Query Methods tùy chỉnh của riêng mình trong interface kế thừa từ `CrudRepository`.

4.4.3 PagingAndSortingRepository<T, ID>

Kế thừa từ `CrudRepository`, `PagingAndSortingRepository` bổ sung các phương thức để hỗ trợ phân trang (pagination) và sắp xếp (sorting) dữ liệu. Điều này rất hữu ích khi làm việc với các tập dữ liệu lớn.

Java

```

public interface UserRepository extends PagingAndSortingRepository<User,
Long> {
    // Kế thừa tất cả các method từ CrudRepository cộng thêm:
    // findAll(Sort sort): Trả về tất cả các Entity đã được sắp xếp theo tiêu
chí Sort.
    // findAll(Pageable pageable): Trả về một Page các Entity, hỗ trợ phân

```



```
trang và sắp xếp.  
}
```

Giải thích:

- `PagingAndSortingRepository` là lựa chọn lý tưởng khi bạn cần hiển thị dữ liệu theo từng trang hoặc cần sắp xếp dữ liệu theo một thứ tự cụ thể.
- `Sort` : Đối tượng `Sort` cho phép bạn định nghĩa các tiêu chí sắp xếp (ví dụ: sắp xếp theo tên tăng dần, theo tuổi giảm dần).
- `Pageable` : Đối tượng `Pageable` chứa thông tin về số trang, kích thước trang và tiêu chí sắp xếp. Khi bạn gọi `findAll(Pageable pageable)`, Spring Data JPA sẽ trả về một đối tượng `Page` chứa danh sách các Entity cho trang hiện tại, cùng với thông tin về tổng số trang và tổng số phần tử.

4.4.4 JpaRepository<T, ID>

Là interface đầy đủ nhất trong hệ thống phân cấp của Spring Data JPA, kế thừa tất cả các phương thức từ `PagingAndSortingRepository` và `CrudRepository`, đồng thời bổ sung thêm các phương thức đặc thù của JPA.

Java

```
public interface UserRepository extends JpaRepository<User, Long> {  
    // Kế thừa tất cả các method từ PagingAndSortingRepository và  
    // CrudRepository cộng thêm:  
    // flush(): Đồng bộ hóa tất cả các thay đổi đang chờ xử lý trong  
    // Persistence Context với cơ sở dữ liệu.  
    // saveAndFlush(T entity): Lưu một Entity và ngay lập tức đồng bộ hóa  
    // thay đổi với cơ sở dữ liệu.  
    // deleteInBatch(Iterable<T> entities): Xóa một tập hợp các Entity trong  
    // một batch operation (hiệu quả hơn khi xóa nhiều).  
    // deleteAllInBatch(): Xóa tất cả các Entity trong một batch operation.  
    // getOne(ID id) - deprecated, use getReferenceById(ID id): Lấy một tham  
    // chiếu (proxy) đến Entity mà không tải toàn bộ dữ liệu.  
    // getReferenceById(ID id): Tương tự getOne, trả về một tham chiếu đến  
    // Entity.  
  
    // Custom methods  
    List<User> findByNameContaining(String name); // Tìm User có tên chứa một  
    chuỗi con.
```

```
@Query("SELECT u FROM User u WHERE u.age > :age") // Định nghĩa một truy  
vấn JPQL tùy chỉnh.  
List<User> findUsersOlderThan(@Param("age") int age); // Tìm User có tuổi  
lớn hơn giá trị age được truyền vào.  
}
```

Giải thích:

- `JpaRepository` là lựa chọn phổ biến nhất cho hầu hết các trường hợp sử dụng vì nó cung cấp đầy đủ các chức năng CRUD, phân trang, sắp xếp và các thao tác đặc thù của JPA.
- Các phương thức như `flush()` và `saveAndFlush()` rất hữu ích khi bạn cần đảm bảo rằng các thay đổi được ghi vào cơ sở dữ liệu ngay lập tức, ví dụ trong các tình huống cần kiểm tra tính toàn vẹn dữ liệu ngay sau khi lưu.
- `deleteInBatch()` và `deleteAllInBatch()` cung cấp cách xóa dữ liệu hiệu quả hơn bằng cách thực hiện các thao tác xóa theo lô (batch), giảm số lượng truy vấn đến cơ sở dữ liệu.
- `getReferenceById()` (thay thế cho `getOne()`) trả về một proxy của Entity. Proxy này chỉ tải dữ liệu thực sự từ cơ sở dữ liệu khi bạn truy cập vào các thuộc tính của nó, giúp tối ưu hiệu suất trong một số trường hợp.
- Bạn có thể kết hợp `JpaRepository` với các Query Methods và `@Query` annotation để tạo ra các truy vấn phức tạp hơn.

4.5 @EnableJpaRepositories

`@EnableJpaRepositories` là một annotation được sử dụng trong cấu hình Spring để kích hoạt tính năng quét (scanning) các interface repository của Spring Data JPA. Nó cho phép Spring tìm và đăng ký các repository của bạn như các Spring beans.

Java

```
@Configuration // Đánh dấu lớp này là một lớp cấu hình Spring.  
@EnableJpaRepositories( // Kích hoạt tính năng quét các JPA repositories.  
    basePackages = "com.example.repository", // Chỉ định gói (package) mà  
    Spring sẽ quét để tìm các interface repository.  
    entityManagerFactoryRef = "entityManagerFactory", // Tên của bean  
    EntityManagerFactory được sử dụng bởi các repository.  
    transactionManagerRef = "transactionManager" // Tên của bean  
    PlatformTransactionManager được sử dụng cho các transaction của repository.
```

```

)
public class JpaConfig {
    // Các bean cấu hình khác liên quan đến JPA, ví dụ DataSource,
    EntityManagerFactory, TransactionManager.
}

```

Giải thích:

- Trong các ứng dụng Spring truyền thống (không phải Spring Boot), bạn thường cần sử dụng `@EnableJpaRepositories` để báo cho Spring biết nơi tìm các interface repository của bạn.
- `basePackages` : Đây là thuộc tính quan trọng nhất, nó chỉ định một hoặc nhiều gói mà Spring sẽ quét để tìm các interface kế thừa từ `Repository` (hoặc các interface con của nó như `JpaRepository`).
- `entityManagerFactoryRef` và `transactionManagerRef` : Các thuộc tính này được sử dụng để chỉ định tên của các bean `EntityManagerFactory` và `PlatformTransactionManager` mà các repository sẽ sử dụng. Điều này đặc biệt hữu ích khi bạn có nhiều `DataSource` hoặc cần tùy chỉnh cách quản lý transaction.
- **Lưu ý:** Với Spring Boot, `@EnableJpaRepositories` thường không cần thiết vì Spring Boot tự động cấu hình và quét các repository trong gói gốc của ứng dụng (và các gói con của nó) khi bạn sử dụng `spring-boot-starter-data-jpa` .

4.6 Multiple DataSources

Trong một số trường hợp, ứng dụng của bạn có thể cần kết nối đến nhiều cơ sở dữ liệu khác nhau. Spring Boot và Spring Data JPA hỗ trợ cấu hình nhiều `DataSource` và `EntityManagerFactory` .

Java

```

@Configuration // Đánh dấu lớp này là một lớp cấu hình Spring.
public class DatabaseConfig {

    @Primary // Đánh dấu đây là DataSource chính, sẽ được sử dụng mặc định
    nếu không có chỉ định cụ thể.
    @Bean(name = "primaryDataSource") // Định nghĩa một Spring bean có tên
    "primaryDataSource".

```

```

@ConfigurationProperties(prefix = "spring.datasource.primary") // Liên
kết các thuộc tính cấu hình bắt đầu bằng "spring.datasource.primary" với
DataSource này.
public DataSource primaryDataSource() {
    return DataSourceBuilder.create().build(); // Xây dựng DataSource từ
các thuộc tính cấu hình.
}

@Bean(name = "secondaryDataSource") // Định nghĩa một Spring bean có tên
"secondaryDataSource".
@ConfigurationProperties(prefix = "spring.datasource.secondary") // Liên
kết các thuộc tính cấu hình bắt đầu bằng "spring.datasource.secondary" với
DataSource này.
public DataSource secondaryDataSource() {
    return DataSourceBuilder.create().build();
}

@Primary // Đánh dấu đây là EntityManagerFactory chính.
@Bean(name = "primaryEntityManagerFactory") // Định nghĩa một Spring bean
có tên "primaryEntityManagerFactory".
public LocalContainerEntityManagerFactoryBean
primaryEntityManagerFactory(
    EntityManagerFactoryBuilder builder,
    @Qualifier("primaryDataSource") DataSource dataSource) { // Sử
dụng @Qualifier để inject primaryDataSource.
    return builder
        .dataSource(dataSource) // Gán DataSource cho
EntityManagerFactory.
        .packages("com.example.primary.entity") // Chỉ định gói chứa
các Entity cho Persistence Unit này.
        .persistenceUnit("primary") // Đặt tên cho Persistence Unit.
        .build();
}

@Bean(name = "secondaryEntityManagerFactory") // Định nghĩa một Spring
bean có tên "secondaryEntityManagerFactory".
public LocalContainerEntityManagerFactoryBean
secondaryEntityManagerFactory(
    EntityManagerFactoryBuilder builder,
    @Qualifier("secondaryDataSource") DataSource dataSource) { // Sử
dụng @Qualifier để inject secondaryDataSource.
    return builder
        .dataSource(dataSource)
        .packages("com.example.secondary.entity") // Chỉ định gói
chứa các Entity cho Persistence Unit thứ cấp.
        .persistenceUnit("secondary") // Đặt tên cho Persistence Unit
thứ cấp.
        .build();
}

```

```
}  
}
```

Giải thích:

- **@Configuration** : Đánh dấu lớp này là một lớp cấu hình Spring, nơi bạn định nghĩa các bean.
- **@Bean** : Đánh dấu một phương thức sẽ tạo ra một Spring bean.
- **@Primary** : Khi có nhiều bean cùng loại, **@Primary** chỉ định bean nào sẽ được ưu tiên inject nếu không có chỉ định cụ thể.
- **@ConfigurationProperties** : Cho phép bạn liên kết các thuộc tính từ file cấu hình (như `application.properties` hoặc `application.yml`) với các thuộc tính của một bean. Ví dụ, `spring.datasource.primary.url`, `spring.datasource.primary.username`, v.v., sẽ được tự động ánh xạ vào `primaryDataSource`.
- **DataSourceBuilder.create().build()** : Một cách tiện lợi để tạo `DataSource` từ các thuộc tính cấu hình.
- **LocalContainerEntityManagerFactoryBean** : Bean này chịu trách nhiệm tạo ra `EntityManagerFactory` của JPA. `EntityManagerFactory` là một đối tượng nặng, thường chỉ có một instance cho mỗi Persistence Unit trong ứng dụng.
- **EntityManagerFactoryBuilder** : Một helper class được Spring Boot cung cấp để dễ dàng cấu hình `EntityManagerFactory`.
- **@Qualifier** : Được sử dụng để phân biệt và chọn đúng bean khi có nhiều bean cùng loại (ví dụ: `primaryDataSource` và `secondaryDataSource`).
- **packages** : Chỉ định gói (package) nơi các Entity của Persistence Unit này được định nghĩa. Điều này giúp JPA biết những lớp nào cần được ánh xạ tới cơ sở dữ liệu.
- **persistenceUnit** : Đặt tên cho Persistence Unit. Tên này sẽ được sử dụng để tham chiếu đến `EntityManagerFactory` cụ thể, ví dụ khi sử dụng `@PersistenceContext(unitName = "secondary")`.

4.7 @Transactional

`@Transactional` là một annotation mạnh mẽ trong Spring Framework, cho phép bạn quản lý transaction một cách khai báo (declarative transaction management). Khi một phương thức được đánh dấu bằng `@Transactional`, Spring sẽ tự động bắt đầu một transaction trước khi phương thức được thực thi và commit hoặc rollback transaction sau khi phương thức hoàn thành, tùy thuộc vào kết quả của nó.

Java

```
@Service // Đánh dấu lớp này là một Spring Service.
@Transactional // Áp dụng transaction cho tất cả các phương thức công khai trong lớp này.
public class UserService {

    @Autowired // Tự động inject UserRepository vào UserService.
    private UserRepository userRepository;

    // Method này sẽ chạy trong một transaction. Nếu có lỗi xảy ra, transaction sẽ được rollback.
    public User createUser(User user) {
        return userRepository.save(user);
    }

    // Read-only transaction: Tối ưu hóa cho các thao tác chỉ đọc, không thay đổi dữ liệu.
    @Transactional(readOnly = true)
    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    // Custom transaction settings: Tùy chỉnh các thuộc tính của transaction.
    @Transactional(
        propagation = Propagation.REQUIRES_NEW, // Tạo một transaction mới, độc lập với transaction hiện có (nếu có).
        isolation = Isolation.READ_COMMITTED, // Mức độ cô lập transaction: chỉ đọc dữ liệu đã được commit.
        timeout = 30, // Thời gian tối đa (giây) cho transaction hoàn thành. Nếu quá thời gian, transaction sẽ bị rollback.
        rollbackFor = Exception.class // Chỉ định các loại ngoại lệ sẽ kích hoạt rollback transaction.
    )
    public void complexOperation() {
        // Logic nghiệp vụ phức tạp cần quản lý transaction chặt chẽ.
    }
}
```

```
}  
}
```

Giải thích:

- **Declarative Transaction Management:** Thay vì phải viết code `try-catch-finally` để quản lý transaction thủ công, `@Transactional` cho phép bạn chỉ cần đánh dấu các phương thức hoặc lớp cần quản lý transaction. Spring AOP (Aspect-Oriented Programming) sẽ "can thiệp" và thêm logic quản lý transaction vào thời điểm chạy.
- **readOnly = true** : Khi một transaction được đánh dấu là `readOnly` , JPA/Hibernate có thể áp dụng các tối ưu hóa hiệu suất, ví dụ như không cần kiểm tra các thay đổi của Entity để đồng bộ hóa với cơ sở dữ liệu.
- **propagation** : Xác định cách các transaction được lan truyền khi một phương thức được gọi từ một phương thức khác đã có transaction. Các giá trị phổ biến:
 - **REQUIRED** (mặc định): Nếu đã có transaction, sử dụng transaction đó. Nếu không, tạo một transaction mới.
 - **REQUIRES_NEW** : Luôn tạo một transaction mới, và nếu đã có transaction, transaction hiện tại sẽ bị tạm dừng.
 - **SUPPORTS** : Nếu đã có transaction, sử dụng transaction đó. Nếu không, chạy mà không có transaction.
- **isolation** : Xác định mức độ cô lập của transaction, ảnh hưởng đến cách các transaction khác nhìn thấy dữ liệu đang được xử lý. Các mức độ phổ biến:
 - **READ_COMMITTED** : Chỉ đọc dữ liệu đã được commit. Ngăn chặn "dirty reads".
 - **READ_UNCOMMITTED** : Có thể đọc dữ liệu chưa được commit (dirty reads).
 - **REPEATABLE_READ** : Đảm bảo rằng các truy vấn lặp lại trong cùng một transaction sẽ trả về cùng một tập hợp dữ liệu. Ngăn chặn "non-repeatable reads".
 - **SERIALIZABLE** : Mức độ cô lập cao nhất, đảm bảo rằng các transaction được thực thi tuần tự. Ngăn chặn "phantom reads".

- **timeout** : Thời gian tối đa (tính bằng giây) mà transaction có thể chạy trước khi bị rollback.
- **rollbackFor** / **noRollbackFor** : Cho phép bạn chỉ định các loại ngoại lệ nào sẽ kích hoạt rollback transaction (**rollbackFor**) hoặc không kích hoạt rollback (**noRollbackFor**). Mặc định, Spring sẽ rollback cho các **RuntimeException** và **Error** , nhưng không rollback cho các **Checked Exception** .

4.8 Exception Translation

Spring tự động chuyển đổi các ngoại lệ cụ thể của JPA (như **PersistenceException** , **NoResultException** , **ConstraintViolationException**) thành các ngoại lệ trừu tượng và dễ quản lý hơn của Spring (**DataAccessException** hierarchy). Điều này giúp ứng dụng của bạn độc lập hơn với công nghệ persistence cụ thể và dễ dàng xử lý lỗi hơn.

Java

```
@Repository // Đánh dấu lớp này là một Repository, kích hoạt tính năng dịch
ngoại lệ của Spring.
public class UserRepositoryImpl {

    @PersistenceContext
    private EntityManager entityManager;

    public User findByEmail(String email) {
        try {
            return entityManager.createQuery(
                "SELECT u FROM User u WHERE u.email = :email", User.class)
                .setParameter("email", email)
                .getSingleResult(); // Phương thức này sẽ ném
// NoResultException nếu không tìm thấy kết quả.
        } catch (NoResultException e) {
            // Spring sẽ tự động wrap NoResultException thành
// EmptyResultDataAccessException (một loại DataAccessException).
            // Bạn không cần phải bắt và ném lại ngoại lệ này một cách thủ
// công nếu đã có @Repository.
            throw e; // Ném lại ngoại lệ để nó được Spring dịch.
        }
    }
}
```

Giải thích:

- Khi bạn sử dụng `@Repository` trên một lớp DAO, Spring sẽ tự động áp dụng một `PersistenceExceptionTranslationPostProcessor` để dịch các ngoại lệ. Điều này có nghĩa là nếu một thao tác JPA ném ra một ngoại lệ cụ thể của JPA (ví dụ: `NoResultException` khi không tìm thấy Entity, hoặc `ConstraintViolationException` khi vi phạm ràng buộc cơ sở dữ liệu), Spring sẽ bắt lấy nó và chuyển đổi thành một ngoại lệ phù hợp trong hệ thống phân cấp `DataAccessException` của Spring.
- Ví dụ, `NoResultException` sẽ được dịch thành `EmptyResultDataAccessException`, và `ConstraintViolationException` có thể được dịch thành `DataIntegrityViolationException`.
- Lợi ích chính của tính năng này là bạn có thể viết code xử lý lỗi một cách nhất quán, không phụ thuộc vào loại công nghệ persistence cụ thể (JPA, JDBC, Hibernate, MyBatis, v.v.). Bạn chỉ cần bắt các `DataAccessException` chung của Spring.

4.9 Testing với Spring Boot

Spring Boot cung cấp các annotation tiện lợi để đơn giản hóa việc kiểm thử các thành phần JPA, đặc biệt là các repository. `@DataJpaTest` là một trong số đó, nó tự động cấu hình một môi trường kiểm thử với cơ sở dữ liệu nhúng (in-memory database) và các thành phần JPA cần thiết.

Java

```
@DataJpaTest // Annotation này cấu hình một slice test cho các thành phần
              JPA. Nó sẽ tự động cấu hình một in-memory database và EntityManager.
class UserRepositoryTest {

    @Autowired // Inject TestEntityManager, một phiên bản của EntityManager
              được thiết kế cho việc kiểm thử.
    private TestEntityManager entityManager;

    @Autowired // Inject UserRepository của bạn.
    private UserRepository userRepository;

    @Test // Đánh dấu phương thức này là một test case.
    void testFindByEmail() {
        // Given: Chuẩn bị dữ liệu test.
        User user = new User("John", "john@example.com");
        entityManager.persistAndFlush(user); // Lưu User vào database và đồng
        bộ hóa ngay lập tức để đảm bảo dữ liệu có sẵn cho truy vấn.
```

```
// When: Thực hiện hành động cần kiểm thử.
User found = userRepository.findByEmail("john@example.com"); // Gọi
phương thức của repository.

// Then: Kiểm tra kết quả.
assertThat(found.getName()).isEqualTo("John"); // Sử dụng AssertJ để
kiểm tra xem tên của User tìm được có đúng không.
    }
}
```

Giải thích:

- `@DataJpaTest` : Đây là một annotation kiểm thử chuyên biệt cho các thành phần JPA. Nó chỉ tải các bean liên quan đến JPA (DataSource, EntityManager, Spring Data JPA repositories) và cấu hình một cơ sở dữ liệu nhúng (thường là H2) để kiểm thử nhanh chóng mà không cần tương tác với cơ sở dữ liệu thực.
- `TestEntityManager` : Một phiên bản của `EntityManager` được cung cấp bởi Spring Boot, rất hữu ích cho việc chuẩn bị dữ liệu test (`persistAndFlush`) và kiểm tra dữ liệu sau khi thực hiện các thao tác.
- `@Autowired` : Spring sẽ tự động inject các bean cần thiết vào lớp kiểm thử của bạn.
- **Workflow kiểm thử:** Thông thường, một test case sẽ tuân theo mô hình Given-When-Then:
 - **Given:** Thiết lập trạng thái ban đầu và dữ liệu cần thiết cho test.
 - **When:** Thực hiện hành động hoặc gọi phương thức cần kiểm thử.
 - **Then:** Xác nhận rằng kết quả của hành động là đúng như mong đợi.

4.10 Configuration Properties

Spring Boot cung cấp một loạt các thuộc tính cấu hình cho JPA và Hibernate, cho phép bạn tinh chỉnh hành vi của chúng thông qua file `application.properties` hoặc `application.yml` .

Plain Text

```
# Hibernate DDL - Cấu hình cách Hibernate quản lý schema cơ sở dữ liệu.
spring.jpa.hibernate.ddl-auto=create-drop # Các giá trị phổ biến: create,
```

create-drop, update, validate, none.

create-drop: Tạo schema khi khởi động ứng dụng và xóa khi ứng dụng dừng.

SQL Logging - Cấu hình để hiển thị các câu lệnh SQL và tham số trên console, rất hữu ích cho việc debug.

spring.jpa.show-sql=true # Hiển thị các câu lệnh SQL được thực thi.

spring.jpa.properties.hibernate.format_sql=true # Định dạng SQL cho dễ đọc.

logging.level.org.hibernate.SQL=DEBUG # Bật logging cấp DEBUG cho các câu lệnh SQL của Hibernate.

logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE # Bật logging cấp TRACE để hiển thị các tham số được bind vào câu lệnh SQL.

Naming Strategy - Cấu hình chiến lược đặt tên cho bảng và cột trong cơ sở dữ liệu.

spring.jpa.hibernate.naming.physical-

strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl # Chiến lược đặt tên vật lý: ánh xạ tên thuộc tính Entity sang tên cột/bảng trong DB.

spring.jpa.hibernate.naming.implicit-

strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl # Chiến lược đặt tên ngầm định: xử lý các trường hợp không có ánh xạ rõ ràng.

Connection Pool (HikariCP) - Cấu hình cho HikariCP, connection pool mặc định của Spring Boot.

spring.datasource.hikari.maximum-pool-size=20 # Số lượng kết nối tối đa trong pool.

spring.datasource.hikari.minimum-idle=5 # Số lượng kết nối tối thiểu luôn được giữ trong pool.

spring.datasource.hikari.idle-timeout=300000 # Thời gian (ms) một kết nối có thể ở trạng thái idle trước khi bị đóng.

spring.datasource.hikari.max-lifetime=600000 # Thời gian (ms) tối đa một kết nối có thể tồn tại trong pool.

JPA Properties - Các thuộc tính JPA/Hibernate nâng cao để tối ưu hiệu suất.

spring.jpa.properties.hibernate.jdbc.batch_size=20 # Kích thước batch cho các thao tác JDBC (insert, update, delete), giúp giảm số lượng round-trip đến DB.

spring.jpa.properties.hibernate.order_inserts=true # Sắp xếp các câu lệnh INSERT theo thứ tự để tối ưu batching.

spring.jpa.properties.hibernate.order_updates=true # Sắp xếp các câu lệnh UPDATE theo thứ tự để tối ưu batching.

spring.jpa.properties.hibernate.jdbc.batch_versioned_data=true # Cho phép batching cho dữ liệu có version (ví dụ: với @Version).

Giải thích:

- **spring.jpa.hibernate.ddl-auto** : Kiểm soát việc tạo và cập nhật schema cơ sở dữ liệu. `create-drop` thường được dùng trong môi trường phát triển để đảm bảo một môi trường sạch sẽ cho mỗi lần chạy test hoặc phát triển.
- **SQL Logging**: Các thuộc tính này rất quan trọng cho việc debug và hiểu cách Hibernate tương tác với cơ sở dữ liệu. `show-sql` hiển thị SQL, `format_sql` làm cho nó dễ đọc hơn, và các cấu hình `logging.level` cung cấp thông tin chi tiết hơn về các tham số được bind.
- **Naming Strategy**: Giúp bạn kiểm soát cách các tên Entity và thuộc tính được ánh xạ thành tên bảng và cột trong cơ sở dữ liệu. Điều này quan trọng để duy trì sự nhất quán và tuân thủ các quy ước đặt tên của cơ sở dữ liệu.
- **Connection Pool (HikariCP)**: Cấu hình HikariCP để quản lý hiệu quả các kết nối đến cơ sở dữ liệu, cải thiện đáng kể hiệu suất ứng dụng bằng cách tái sử dụng các kết nối thay vì tạo mới cho mỗi yêu cầu.
- **JPA Properties (Batching)**: Các thuộc tính này liên quan đến việc tối ưu hóa hiệu suất bằng cách sử dụng JDBC batching. Thay vì gửi từng câu lệnh SQL riêng lẻ đến cơ sở dữ liệu, Hibernate có thể nhóm nhiều câu lệnh lại thành một batch và gửi chúng cùng một lúc, giảm thiểu chi phí mạng và cải thiện thông lượng.